

Internal Architecture of 8086

8086 has two blocks BIU and EU. The BIU performs all bus operations such as instruction fetching, reading and writing operands for memory and calculating the addresses of the memory operands. The instruction bytes are transferred to the instruction queue. EU executes instructions from the instruction system byte queue. Both units operate asynchronously to give the 8086 an overlapping instruction fetch and execution mechanism which is called as Pipelining. This results in efficient use of the system bus and system performance. BIU contains Instruction queue, Segment registers, Instruction pointer, and Address adder. EU contains Control circuitry, Instruction decoder, ALU, Pointer, Index register and Flag register.

Bus Interface Unit:

It provides a full 16 bit bidirectional data bus and 20 bit address bus. The bus interface unit is responsible for performing all external bus operations.

Instruction fetches Instruction queuing, Operand fetch and storage, Address relocation and Bus control. The BIU uses a mechanism known as an instruction stream queue to implement pipeline architecture. This queue permits prefetching of up to six bytes of instruction code. Whenever the queue of the BIU is not full, it has room for at least two more bytes and at the same time the EU is not requesting it to read or write operands from memory, the BIU is free to look ahead in the program by prefetching the next sequential instruction. These prefetching instructions are held in its FIFO queue. With its 16 bit data bus, the BIU fetches two instruction bytes in a single memory cycle. After a byte is loaded at the input end of the queue, it automatically shifts up through the FIFO to the empty location nearest the output.

The EU accesses the queue from the output end. It reads one instruction byte after the other from the output of the queue. If the queue is full and the EU is not requesting access to operand in memory, these intervals of no bus activity, which may occur between bus cycles, are known as Idle state. If the BIU is already in the process of fetching an instruction when the EU request it to read or write operands from memory or I/O, the BIU first completes the instruction fetch bus cycle before initiating the operand read / write cycle. The BIU also contains a dedicated adder which is used to generate the 20bit physical address that is output on the address bus. This address is formed by adding an appended 16 bit segment address and a 16 bit offset address. For example: The physical address of the next instruction to be fetched is formed by combining the current contents of the code segment CS register and the current contents of the instruction pointer IP register. The BIU is also responsible for generating bus control signals such as those for memory read or write and I/O read or write.

Execution Unit

The Execution unit is responsible for decoding and executing all instructions. The EU extracts instructions from the top of the queue in the BIU, decodes them, generates operands if necessary, passes them to the BIU and requests it to perform the read or write bus cycles to memory or I/O and perform the operation specified by the instruction on the operands. During the execution of the instruction, the EU tests the status and control flags and updates them based on the results of executing the instruction. If the queue is empty, the EU waits for the next instruction byte to be fetched and shifted to top of the queue. When the EU executes a branch or jump instruction, it transfers control to a location corresponding to another set of

sequential instructions. Whenever this happens, the BIU automatically resets the queue and then begins to fetch instructions from this new location to refill the queue.

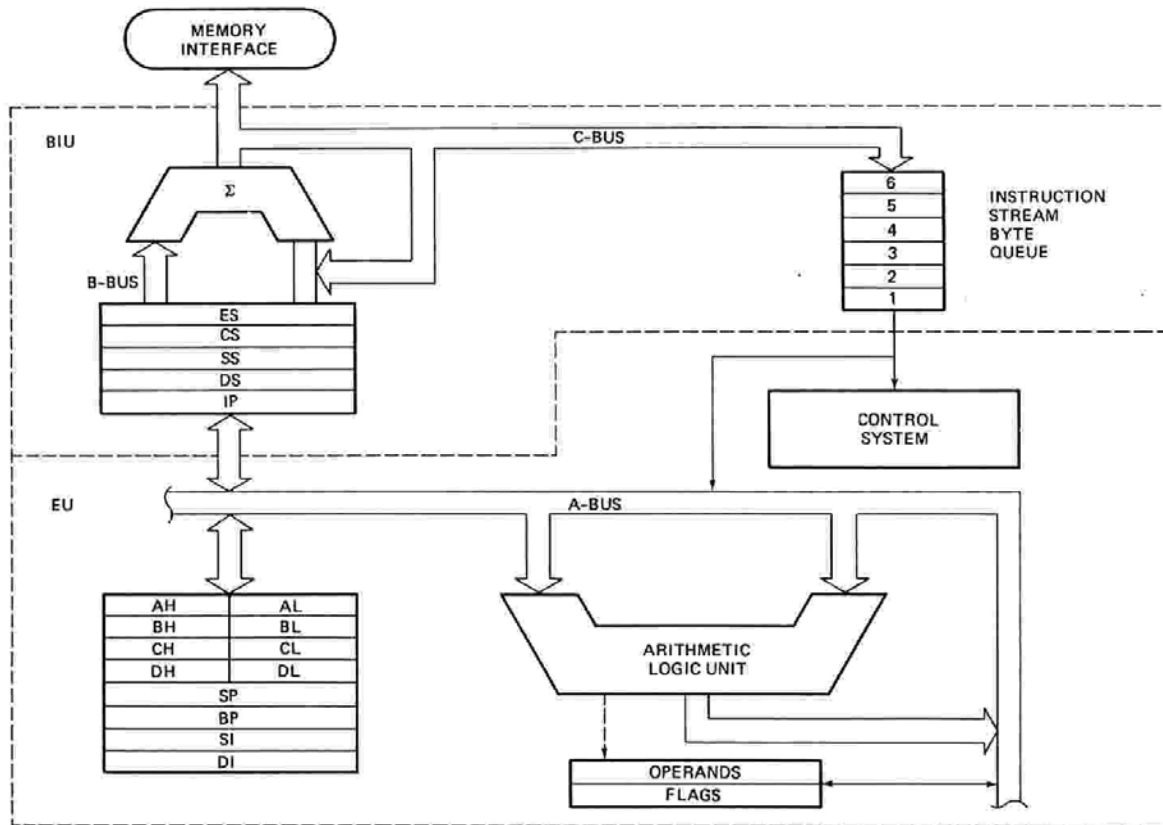


FIGURE 2-7 8086 internal block diagram. (Intel Corp.)

Figure 1 : Block Diagram of 8086

Internal Registers of 8086

The 8086 has four groups of the user accessible internal registers. They are the instruction pointer, four data registers, four pointer and index register, four segment registers. The 8086 has a total of fourteen 16-bit registers including a 16-bit register called the status register, with 9 of bits implemented for status and control flags. Most of the registers contain data/instruction offsets within 64 KB memory segment. There are four different 64 KB segments for instructions, stack, data and extra data. To specify where in 1 MB of processor memory these 4 segments are located the processor uses four segment registers:

Code segment (CS) is a 16-bit register containing address of 64 KB segment with processor instructions. The processor uses CS segment for all accesses to instructions referenced by instruction pointer (IP) register. CS register cannot be changed directly. The CS register is automatically updated during far jump, far call and far return instructions. It contains the starting address of a program's code segment. This segment address plus an offset value in the IP register indicates the address of an instruction to be fetched for execution.

Stack segment (SS) is a 16-bit register containing address of 64KB segment with program stack. It permits the implementation of a stack in memory. By default; the processor assumes that all data referenced by the stack pointer (SP) and base pointer (BP) registers is located in the stack segment. It stores the starting

address of a program's stack segment the SS register. SS register can be changed directly using POP instruction.

Data segment (DS) is a 16-bit register containing address of 64KB segment with program data. By default, the processor assumes that all data referenced by general registers (AX, BX, CX, and DX) and index register (SI, DI) is located in the data segment. It contains the starting address of a program's data segment. Instruction uses this address to locate data. This address plus an offset value in an instruction causes a reference to a specific byte location in the data segment. DS register can be changed directly using POP and LDS instructions.

Extra Segment (ES) is a 16-bit register containing address of 64KB segment, usually with program data.

By default, the processor assumes that the DI register points to the ES segment in string manipulation Instructions. ES register can be changed directly using POP and LES instructions.. It is used by some string operations to handle memory addressing.

Accumulator register consists of two 8-bit registers AL and AH, which can be combined together and used as a 16-bit register AX. AL in this case contains the low order byte of the word, and AH contains the high-order byte. Accumulator can be used for I/O operations and string manipulation.

Base register consists of two 8-bit registers BL and BH, which can be combined together and used as a 16-bit register BX. BL in this case contains the low-order byte of the word, and BH contains the high-order byte. BX register usually contains a data pointer used for based, based indexed or register indirect addressing.

Count register consists of two 8-bit registers CL and CH, which can be combined together and used as a 16-bit register CX. When combined, CL register contains the low order byte of the word, and CH contains the high-order byte. Count register can be used in Loop, shift/rotate instructions and as a counter in string manipulation,.

Data register consists of two 8-bit registers DL and DH, which can be combined together and used as a 16-bit register DX. When combined, DL register contains the low order byte of the word, and DH contains the high-order byte. Data register can be used as a port number in I/O operations. In integer 32-bit multiply and divide instruction the DX register contains high-order word of the initial or resulting number.

The following registers are both general and index registers:

POINTER REGISTERS

The 16 bit Pointer Registers are IP, SP and BP respectively. SP and BP are located in EU whereas IP is located in BIU

Stack Pointer (SP) is a 16-bit register pointing to program stack. The 16 bit SP Register provides an offset value, which when associated with the SS register (SS: SP)

Base Pointer (BP) is a 16-bit register pointing to data in stack segment. The 16 bit BP facilitates referencing parameters, which are data and addresses that a program passes via the stack. The processor combines the addresses in SS with the offset in BP. BP can also be combined with DI and SI as a base register for special addressing. BP register is usually used for based, based indexed or register indirect addressing.

INDEX REGISTERS

The 16 bit Index Registers are SI and DI

Source Index (SI) is a 16-bit register. SI is used for indexed, based indexed and register indirect addressing, as well as a source data addresses in string manipulation instructions. SI is associated with the DS Register.

Destination Index (DI) is a 16-bit register. DI is used for indexed, based indexed and register indirect addressing, as well as a destination data addresses in string manipulation instructions. In this context, DI is associated with the ES register.

Other registers:

Instruction Pointer (IP) is a 16-bit register. The 16 bit IP Register contains the offset address of the next instruction that is to execute .IP is associated with CS register as (CS:IP) .For each instruction that executes, the processor changes the offset value in IP so that IP in effect directs each step of execution.

The Queue

While the EU is decoding an instruction or executing an instruction which does not require use of the buses, the BIU fetches up to six instructions bytes for the following instructions. The BIU Stores pre-fetched bytes in First in First out register set called a queue. When the EU is ready for its next instruction, it simply reads the instruction bytes for the instruction from the queue in the BIU. This is much faster than sending out an address to the system memory and waiting for memory to send back the next instruction bytes or bytes. Fetching the next instruction while the current instruction executes is called pipelining.

Flags are a 16-bit register containing 9 one bit flags.

Overflow Flag (OF) - set if the result is too large positive number, or is too small negative number to fit into destination operand.

Direction Flag (DF) - if it is set then string manipulation instructions will auto-decrement index registers. If cleared then the index registers will be auto-incremented.

Interrupt-enable Flag (IF) - setting this bit enables maskable interrupts.

Single-step Flag (TF) - if set then single-step interrupt will occur after the next instruction.

Sign Flag (SF) - set if the most significant bit of the result is set.

Zero Flag (ZF) - set if the result is zero.

Auxiliary carry Flag (AF) - set if there was a carry from or borrow to bits 0-3 in the AL register.

Parity Flag (PF) - set if parity (the number of "1" bits) in the low-order byte of the result is even.

Carry Flag (CF) - set if there was a carry from or borrow to the most significant bit during last result calculation.

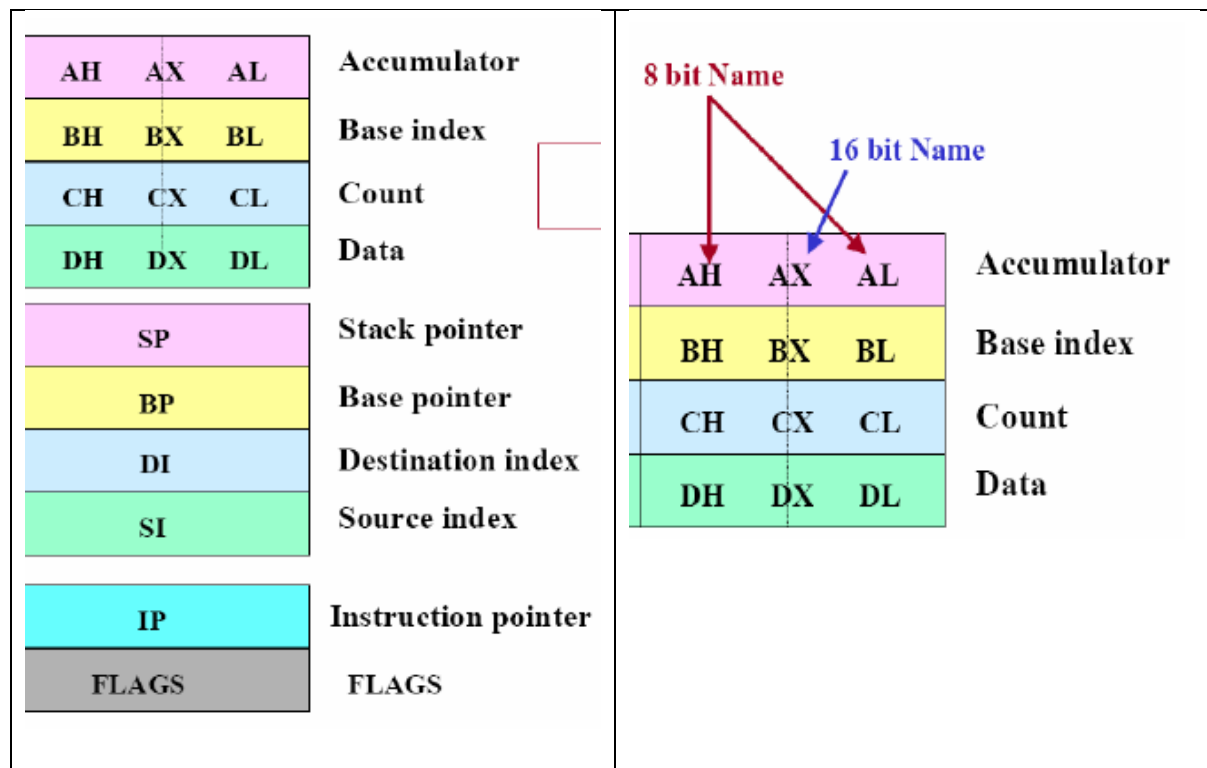


Figure 2 : Internal Registers of 8086

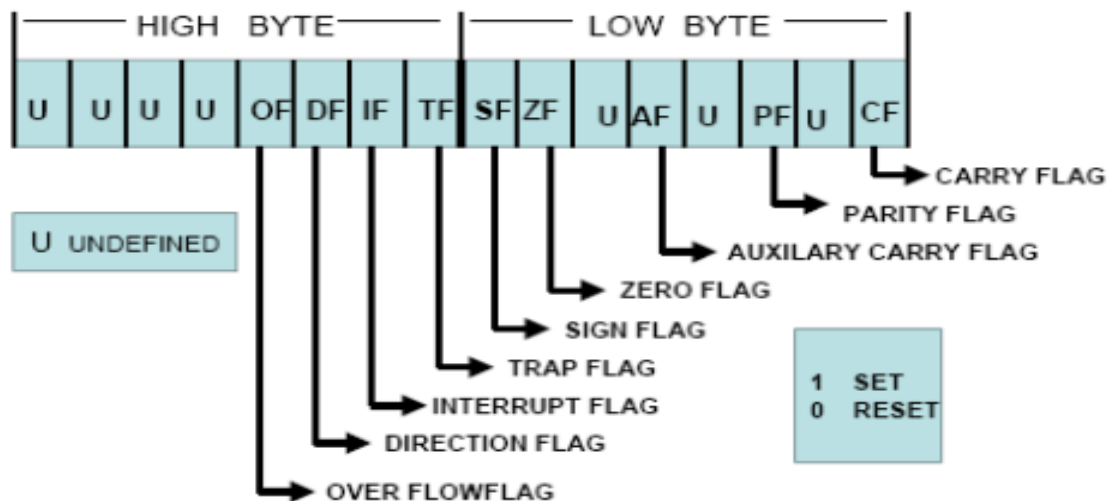


Figure 3 : Flag register of 8086

CONDITIONAL FLAGS

CF = CARRY FLAG [Set by Carry out of MSB]

PF = PARITY FLAG [Set if Result has even parity]

AF= AUXILIARY CARRY FLAG FOR BCD

ZF = ZERO FLAG [Set if Result is 0]

SF = SIGN FLAG [MSB of Result]

OF = OVERFLOW FLAG

CONTROL FLAG

TF = SINGLE STEP TRAP FLAG

IF = INTERRUPT ENABLE FLAG

DF = STRING DIRECTION FLAG

Memory Organization

INTRODUCTION

The processor provides a 20-bit address to memory, which locates the byte being referenced. The memory is organized as a linear array of up to 1 million bytes, addressed as 00000(H) to FFFFF (H). The memory is logically divided into code, data, and extra and stacks segments of up to 64k bytes each, with each segment falling on 16-byte boundaries (see Figure). All memory references are made relative to base addresses contained in high speed segment registers. The segment types were chosen based on the addressing needs of programs. All information in one segment type shares the same logical attributes (e.g. code or data). By structuring memory into re-locatable areas of similar characteristics and by automatically selecting segment registers, programs are shorter, faster and more structured.

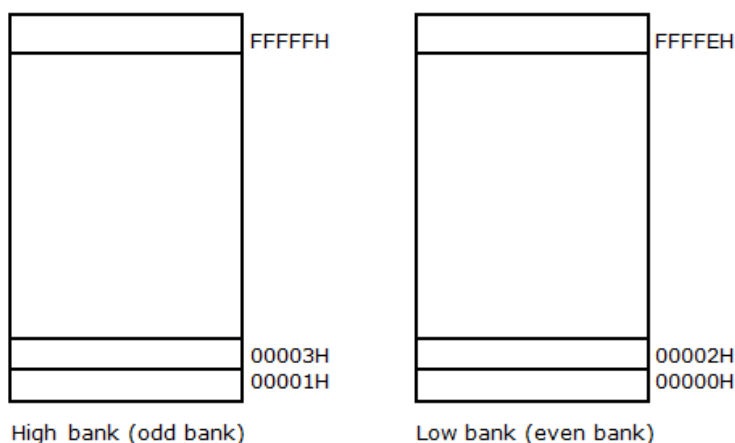


Figure: The Physical Memory System of 8086

The Intel 8086 is a 16 bit Microprocessor that is intended to be used as the CPU in a Microcomputer. The 8086 has a 20 bit address bus so it can address any one of 2^{20} or 1,048,576 memory locations. Each of the 1,048,576 memory address of the 8086 represents a byte-wide location.

16 bit word will be stored in two consecutive memory locations. If the first byte of a word is at an even address, the 8086 can read the entire word in one operation. If the first byte of a word is at an odd address, the 8086 will read the first byte with one bus cycle and the second byte with another bus cycle.

ACCESSING DATA IN MEMORY

An important point here is that an 8086 always stores the low byte of word in lower address and stores high byte of word in higher address. It follows little Endian format .

Low Byte – Low Address: High Byte – High Address (Little Endian)

- As an example, suppose we have the hexadecimal number 12345678.
- The big endian and little endian arrangements of the bytes are shown below.

Address →	00	01	10	11
Big Endian	12	34	56	78
Little Endian	78	56	34	12

CALCULATING PHYSICAL ADDRESS (Segment Arithmetic)

To perform segment arithmetic successfully, it helps to understand how the processor combines a 16-bit segment and a 16-bit offset to form a 20-bit linear address. In effect, the segment selects a 64K region of memory, and the offset selects the byte within that region. Here it is shown how it works:

1. The processor shifts the segment address to the left by four binary places, producing a 20-bit address ending in four zeros. This operation has the effect of multiplying the segment address by 16.
2. The processor adds this 20-bit segment address to the 16-bit offset address. The offset address is not shifted.
3. The processor uses the resulting 20-bit address, called the "physical address," to access an actual location in the 1-megabyte address space.

Figure below illustrates this process.

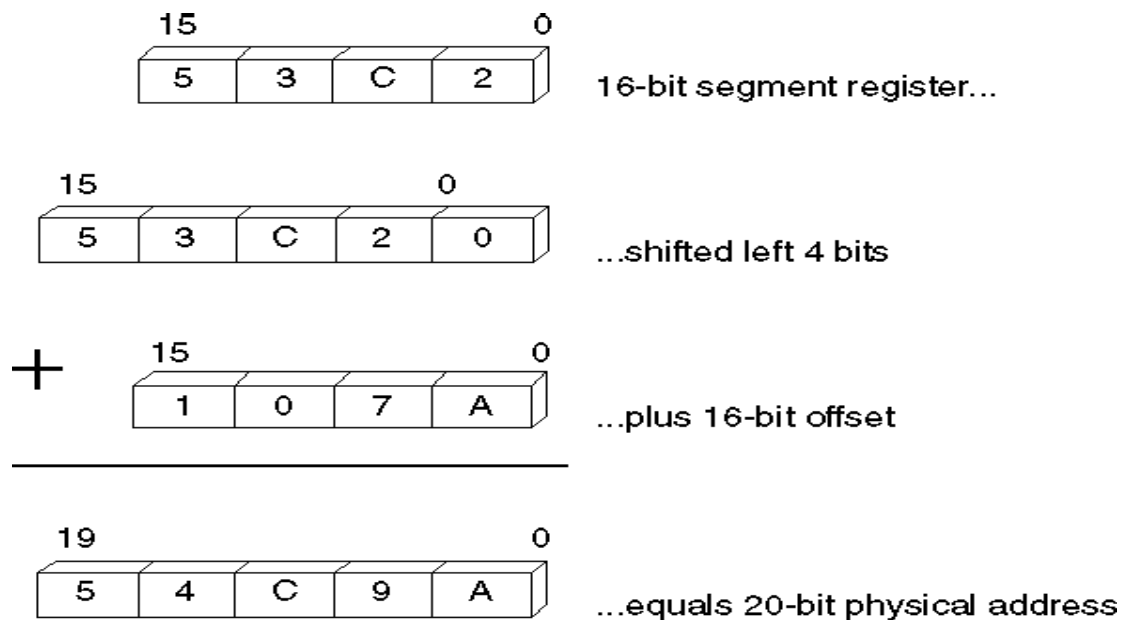


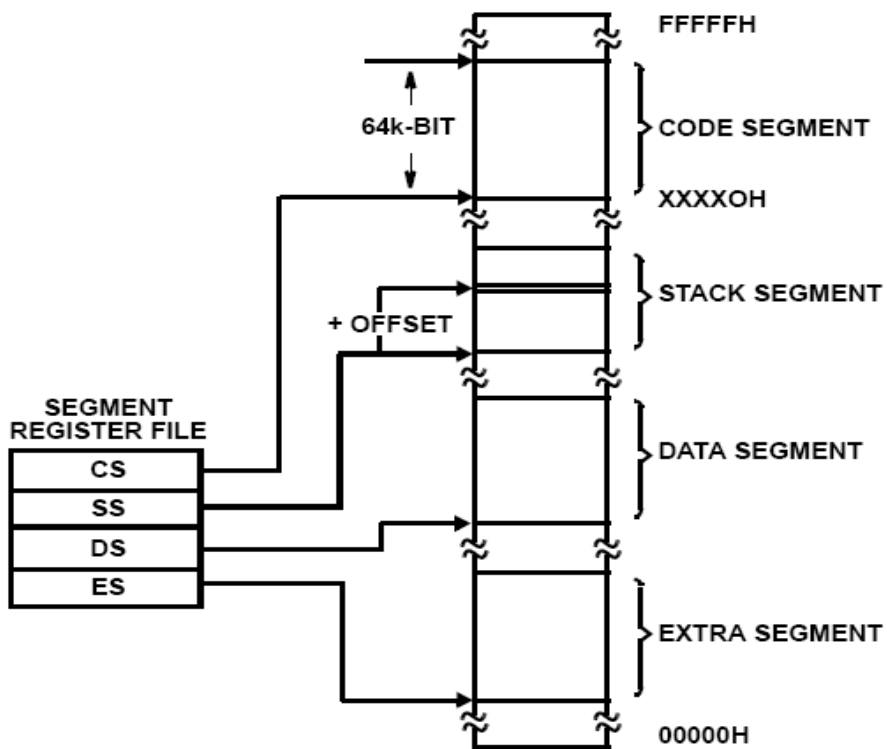
Figure: Calculating Physical Addresses

A 20-bit physical address may actually be specified by 4,096 equivalent *segment: offset* addresses. For example, the addresses 0000:F800, 0F00:0800, and 0F80:0000 all refer to the same physical address 0F800.

SEGMENTED MEMORY

The 8086 family of processors employs a segmented architecture - that is, each address is represented as a segment and an offset. Segmented addresses affect many aspects of assembly-language programming, especially addresses and pointers. Segmented architecture was originally designed to enable a 16-bit processor to access an address space larger than 64K. (The section "Segmented Addressing," later in this chapter, explains how the processor uses both the segment and offset to create addresses larger than 64K.) MS-DOS is an example of an operating system that uses segmented architecture on a 16-bit processor. With the advent of protected-mode processors such as the 80286, segmented architecture gained a second purpose. Segments can separate different blocks of code and data to protect them from undesirable interactions. Windows takes advantage of the protection features of the 16-bit segments on the 80286. Segmented architecture went through another significant change with the release of 32-bit processors, starting with the 80386. These processors are compatible with the older 16-bit processors, but allow flat model 32-bit offset values up to 4 gigabytes. Offset values of this magnitude remove the memory limitations of segmented architecture

The 8086 BIU sends out 20 bit address so it can address any of 2^{20} or 1,048,576 bytes in memory. However at any given time the 8086 works with only four 65536 bytes (64 Kbyte) segment within this 1,048,576 byte (1M Byte) Range .Four segments are: Code Segment, Stack Segment, Data Segment and Extra Segment .Four segment registers in BIU are used to hold the upper 16 bits of the starting address of 4 memory segments that the 8086 is working with at a particular time. The 4 segment registers are code segment register (CS), stack segment register (SS), data segment register (DS) and the extra segment register (ES). For small programs which do not need all 64 Kbytes in each segment can overlap. For example, the code segment holds the upper 16 bits of the starting address for the segment from which the BIU is currently fetching instruction code bytes. The BIU always inserts zero for the lowest 4 bits of the 20 bit starting address. If the code segment register contains 348A H then the code segment will start at address 348A0 H .A 64 Kbytes segment can be located anywhere within the 1 MByte address space, but the segment will always start at an address with zeros in the lowest 4 bits.



ADVANTAGES OF SEGMENTATION [SEGMENT: OFFSET SCHEME]

Intel designed the 8086 family devices to access memory using the segment: offset approach rather than accessing memory directly with 20 bit.

The advantages are listed below.

- The segment: offset scheme requires only a 16 bit number to represent the base address for a segment and only a 16 bit offset to access any location in a segment. This means that 8086 has to manipulate and store only 16 bit quantities instead of 20 bit quantities.
- This makes easier interface with 8 and 16 bit wide memory boards and with 16 bit registers in the 8086.
- It allows programs to be relocated in memory system. A re-locatable program is one that can be placed in any area of memory and executed without change.
- It allows programs written to function in the real mode to operate in protected mode.
- Segmentation also makes easy to keep user's program and data separate from one another and segmentation makes it easy to switch from one user's program to another user's program.

DISADVANTAGE OF SEGMENT: OFFSET APPROACH

The segment: offset scheme needs complex hardware and software i.e. overhead of complex memory management.

SEGMENTS OFFSET COMBINATION

SEGMENT	OFFSET	SPECIAL PURPOSE
CS [CODE SEGMENT]	IP [INSTRUCTION POINTER]	INSTRUCTION ADDRESS
SS [STACK SEGMENT]	SP [STACK POINTER] BP [BASE POINTER]	STACK ADDRESS
DS [DATA SEGMENT]	BX [BASE REGISTER] DI [DESTINATION INDEX] SI [SOURCE INDEX] 8 BIT NUMBER 16 BIT NUMBER	DATA ADDRESS
ES [EXTRA SEGMENT]	DI [DESTINATION INDEX]	STRING DESTINATION ADDRESS